

3D Medical Image Interactive Segmentation

Cherish Jain
22b3937@iitb.ac.in

Abstract

This report describes an implementation of an interactive segmentation framework that uses a Sequential Interaction Memory (SIM), a Dynamic Data Generation (DDG) virtual user, and ConvLSTM-guided decoders. The content and methods implemented are drawn from the referenced paper and the project presentation.

1 Introduction and Previous Work

Interactive medical image segmentation combines automatic models with minimal user corrections to obtain clinically usable results quickly. Manual annotation is time-consuming (10–50 minutes per MRI series) while fully automatic methods cannot yet be trusted for many clinically-critical tasks; interactive methods provide a practical trade-off between automation and expert oversight. Key prior approaches include classical interactive methods (Graph Cuts, Random Walker, GeoS) and modern deep-learning encoder–decoder models adapted for interactive inputs. Prior deep-learning interactive methods typically feed user corrections as additional channels but often use static or cumulative interaction representations that discard temporal ordering. The sequential-memory idea and dynamic click simulation in the 2024 paper provide the main inspiration for the implementation reproduced here.

2 Why this problem should be solved

- Clinical need: segmentation is essential for diagnosis, planning and AR-guidance; manual annotation is too slow and expensive for routine clinical workflows.
- Safety trust: automatic outputs require expert validation — interactive systems let clinicians control final decisions.
- Data characteristics: medical scans are noisy, low-contrast, multi-class, multi-instance and extremely imbalanced (tiny cavity vs. large background). Robust, interaction-aware systems reduce clinician time while preserving high accuracy.
- Practical impact: reducing segmentation time from tens of minutes to a few minutes or seconds-per-slice saves significant human effort and improves adoption.

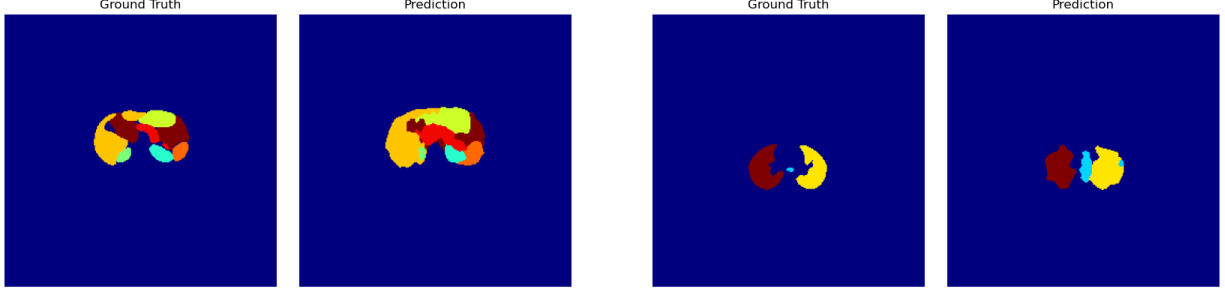


Figure 1: Direct model predictions without interactive refinement. Both examples illustrate baseline segmentation quality before any DDG clicks or SIM updates.

2.1 Interactive Segmentation and the Naive CIM Approach

Interactive segmentation aims to combine automatic model inference with sparse user corrections to obtain accurate medical segmentations with minimal manual effort. Instead of relying solely on a fully automatic model, a user provides corrective interactions—typically foreground/background clicks or scribbles—that guide the model toward the desired boundary. The model is expected to update its prediction after each interaction, gradually reducing errors in regions identified by the user.

A straightforward and widely used strategy for integrating user feedback is the **Cumulative Interaction Memory (CIM)**. In CIM, all user clicks from previous rounds are accumulated and encoded as additional input channels. For a segmentation task with N classes, CIM typically stores one foreground-click channel per class:

$$\text{Input} = I \parallel C_1 \parallel C_2 \parallel \dots \parallel C_N,$$

where I is the image and C_k denotes a binary map highlighting all clicks (up to the current round) for class k .

While simple to implement, CIM suffers from a key limitation: it discards the temporal ordering of interactions. All clicks are merged into static channels regardless of when they were placed. As a result, the model cannot distinguish between early coarse corrections and later fine adjustments, nor can it learn the sequential behaviour typical of real user interaction. This motivates the development of sequential representations such as SIM, which explicitly preserve the order and evolution of interactions.

3 SIM vs CIM

CIM (Cumulative Interaction Memory): merges all previous user corrections into a single set of masks (input channels = $1 + N$). This discards ordering and the temporal signal of how corrections were applied.

SIM (Sequential Interaction Memory): stores D ordered states; each state contains N click masks and N probability maps, so the input channel count becomes $1 + 2DN$. SIM preserves the sequence of corrections and intermediate outputs, allowing the model to learn the refinement trajectory over time.

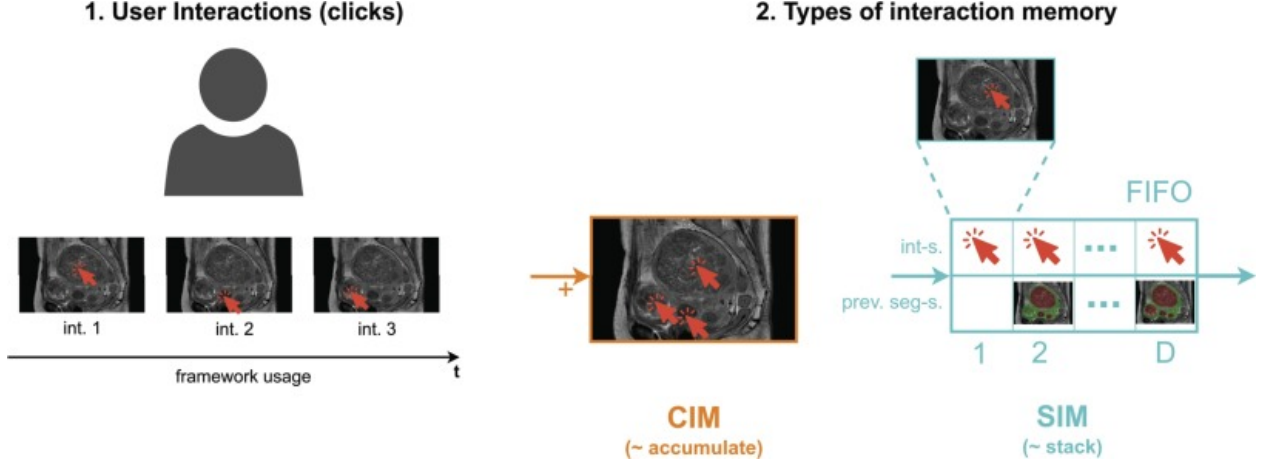


Figure 2: Conceptual comparison between CIM (Click Interaction Maps) and SIM (Sequential Interaction Memory). CIM stores only the latest click map, while SIM preserves a multi-round history of user corrections (clicks + model predictions), enabling more stable and consistent iterative refinement.

Why SIM is preferred:

- Temporal signal: the order in which corrections were made is informative (users react to the current segmentation).
- RNN/ConvLSTM synergy: temporal models (ConvLSTM/LSTM) can exploit sequence information for better refinement.
- Empirical gains: the original paper’s ablation shows DDG-SIM outperforms CIM variants in small and difficult classes (e.g., cavity, tumours).

4 Concept of Virtual User

During training, real users are not available, so a virtual user (DDG) simulates corrective clicks based on prediction–GT disagreement. The virtual user places clicks in mislabelled regions to emulate realistic correction patterns.

5 Baseline Model Details

The baseline architecture implemented in this work is a ResNet34–UNet hybrid equipped with ConvLSTM-based temporal refinement in the decoder. The model is explicitly designed to consume Sequential Interaction Memory (SIM), and to refine features across D interaction rounds. Below we describe the full architecture in detail, corresponding exactly to the implemented code.

5.1 Encoder: ResNet34 with SIM-aware input

The standard ResNet34 backbone expects a 3-channel RGB input. To incorporate sequential interaction memory, we adapt the first convolution to accept:

$$C_{\text{in}} = 1 + 2DN,$$

where 1 is the grayscale image channel, and $2DN$ corresponds to D SIM states each containing an N -channel click map and N -channel probability map.

The first convolution layer is replaced with:

$$\text{Conv2d: } (C_{\text{in}}, 64, 7 \times 7, \text{stride} = 2, \text{padding} = 3),$$

initialized using Kaiming normal initialization.

The rest of the ResNet34 backbone remains unchanged, producing feature maps at four resolutions:

$$\{e_1, e_2, e_3, e_4\} = \{1/4, 1/8, 1/16, 1/32\}.$$

5.2 SIM splitting and multi-scale resizing

Given SIM with shape $(2DN, H, W)$, the model first reshapes it into D temporal states:

$$\text{SIM} \rightarrow \{s_1, s_2, \dots, s_D\}, \quad s_t \in \mathbb{R}^{2N \times H \times W}.$$

Each decoder stage requires SIM tensors at its spatial scale, so every state is resized via bilinear interpolation:

$$s_t^{(k)} = \text{Resize}(s_t, \text{size} = H_k \times W_k),$$

where $k \in \{1/2, 1/4, 1/8, 1/16\}$ corresponds to the decoder levels.

5.3 ConvLSTM cell

Each decoder block uses a ConvLSTM cell defined as:

$$(h_t, c_t) = \text{ConvLSTM}(x_t, h_{t-1}, c_{t-1}),$$

where the gates are computed by a single convolution on the concatenation:

$$[i_t, f_t, o_t, g_t] = \text{Conv2d}([x_t || h_{t-1}]).$$

The recurrent updates follow:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t).$$

Each decoder block initializes the hidden state (h_0, c_0) as zero tensors of appropriate spatial size.

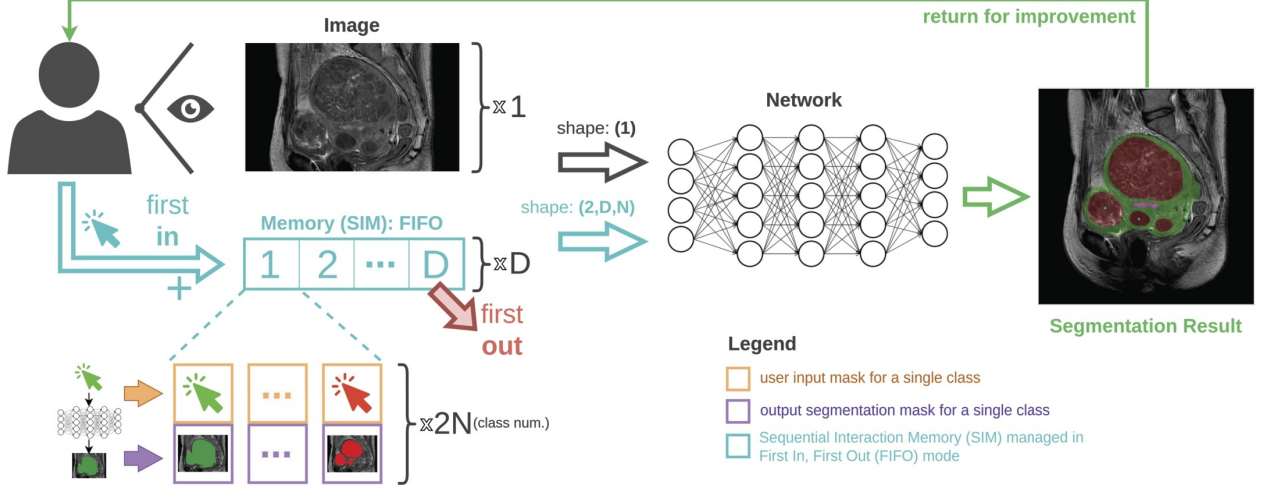


Figure 3: Overall interactive segmentation pipeline: the model receives the CT slice along with the evolving SIM tensor, performs forward prediction, and integrates DDG virtual-user corrections across rounds to iteratively refine the segmentation output.

5.4 Decoder block: upsampling, skip fusion and SIM-guided ConvLSTM

Each decoder block performs:

1. **Upsampling:** transposed convolution or bilinear upsampling:

$$x_{\text{up}} = \text{Upsample}(x_{\text{in}}).$$

2. **Skip concatenation** with encoder features:

$$x = [x_{\text{up}} || e_k].$$

3. **Projection of SIM guidance:** Each SIM state s_t at this scale is projected via a 1×1 convolution:

$$g_t = W_{\text{sim}} * s_t^{(k)}.$$

4. **Guided ConvLSTM refinement:** For each of the D timesteps:

$$x_t = x + g_t, \quad (h_t, c_t) = \text{ConvLSTM}(x_t, h_{t-1}, c_{t-1}).$$

The final hidden state h_D forms the decoder output for that stage.

5. **Post-processing convolution:** A smoothing block (Conv + BN + ReLU) refines h_D .

Thus each decoder block progressively integrates multi-scale image features, skip connections, and temporal correction cues from SIM.

5.5 Full decoder pipeline

The UNet-style decoder contains four ConvLSTM-guided blocks:

$$\text{dec4}, \text{dec3}, \text{dec2}, \text{dec1},$$

corresponding to resolutions $\{1/16, 1/8, 1/4, 1/2\}$ respectively.

After the final stage, one more upsampling is performed:

$$d_0 = \text{ConvTranspose2d}(d_1),$$

which restores full-resolution features.

5.6 Output head

A final 1×1 convolution maps the 64-channel representation to N class logits:

$$\text{logits} = \text{Conv2d}(64, N, 1 \times 1).$$

6 Sequential Interaction Memory (SIM) and Virtual User

6.1 Sequential Interaction Memory (SIM)

We maintain a temporal memory structure containing D interaction rounds, each storing two components:

$$\text{SIM} = \{ M^{(1)}, P^{(1)}, M^{(2)}, P^{(2)}, \dots, M^{(D)}, P^{(D)} \},$$

where $M^{(t)} \in \mathbb{R}^{N \times H \times W}$ is the click mask for round t and $P^{(t)} \in \mathbb{R}^{N \times H \times W}$ is the predicted probability map from that round. Thus SIM has $2DN$ channels in total.

During training, at each new interaction the first unfilled round is detected, and both the click mask and the model's probability map are inserted:

$$\text{SIM}[tN : (t+1)N] \leftarrow M^{(t)}, \quad \text{SIM}[DN + tN : DN + (t+1)N] \leftarrow P^{(t)}.$$

This preserves the order of interactions and provides the decoder with a full refinement history.

6.2 Dynamic Data Generation (DDG) Virtual User

To simulate realistic user corrections, we implement a DDG-based virtual user. At every round:

- Prediction and ground truth are compared to form a disagreement mask.
- Connected components of the mislabelled regions are extracted and sorted by size.
- For each class, up to a maximum number of clicks are placed near the component centre, controlled by a click probability p that decays over rounds.
- A random class may be skipped, and a fraction of clicks are randomly dropped to emulate noisy user behaviour.

The virtual user outputs a one-hot click mask $M^{(t)}$ and updates the click probability as explained in next section. These clicks are written into SIM and guide the network in the subsequent forward pass.

7 Click-placing strategy and the concept of the virtual user

Core DDG rules implemented:

1. Initialize click probability $p \leftarrow 1$. If $p = 1$, place an *initial click* in each connected component (foreground).
2. Otherwise ($p < 1$) place a corrective click in the *largest mislabelled region* (per class) with probability p .
3. After each round, update $p \leftarrow p - \frac{1}{t}$ where t is the maximum number of training interactions.
4. Randomly skip one class per image (to increase variety and reduce interaction dependence).
5. Randomly drop a fixed percentage (80% in the paper; configurable in implementation) of generated clicks to simulate imperfect users.
6. Limit max clicks per component (3) and ensure at least one click is kept when available.

Click localization: clicks are sampled within connected mislabelled components, prioritizing region centers (higher probability near the center) and sorting components by size to prefer larger errors.

8 Improved Model Details

The improved architecture replaces the ResNet34+UNet baseline with a **SwinUNet + bottleneck ConvLSTM + early SIM fusion** design. The goal is to retain the benefits of SIM while reducing recurrent overhead and improving global feature quality.

8.1 Swin Transformer Encoder

We use `swinv2_tiny_window8_256` (via `timm`) as the encoder, producing hierarchical features with channel dimensions [96, 192, 384, 768]. Swin provides long-range context through shifted-window self-attention, yielding stronger representations than CNN-based encoders.

8.2 Early SIM Fusion

Instead of injecting SIM only in the decoder, the entire SIM tensor ($2DN$ channels) is projected into the stage-1 feature dimension and fused with e_1 through a learned spatial attention gate:

$$e_1 \leftarrow e_1 + \sigma(\text{Att}([e_1 || \text{SIM}_{\text{proj}}])) \cdot \text{SIM}_{\text{proj}}.$$

This lets interaction cues influence all deeper Swin layers.

8.3 Single Bottleneck ConvLSTM

Temporal refinement is shifted to a *single* ConvLSTM placed at the bottleneck (e_4). This reduces computational cost compared to running ConvLSTMs at every decoder stage, while still capturing the sequential structure of interactive corrections:

$$(h, c) = \text{ConvLSTM}(e_4, h_{\text{prev}}, c_{\text{prev}}).$$

8.4 Lightweight Decoder

With temporal modeling now isolated at the bottleneck, the decoder becomes a simple upsample+skip-add pathway:

$$d_k = \text{Up}(d_{k+1}) + e_k, \quad k \in \{3, 2, 1\}.$$

A final 1×1 convolution produces segmentation logits. This reduces computation and avoids recurrent loops in the decoder.

8.5 Robust Feature Alignment

Because `timm` backbones may output features in varying shapes, the implementation includes automatic permutation and channel padding/cropping to maintain consistent (B, C, H, W) formats.

9 Dataset

The dataset used in this work was obtained from Tata Memorial Hospital, where all CT scans were manually annotated by expert clinicians. The training split consisted of CT scans from 45 patients, with each patient contributing between 50 and 300 axial slices, resulting in a total of 8064 training slices and 896 validation slices. The test set included 15 additional patients that were never seen during training. The dataset exhibits a strong class imbalance, with the background

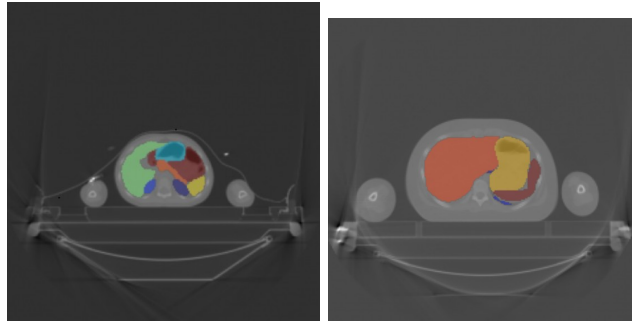


Figure 4: Example annotated CT scan slices from the dataset, showing multi-organ segmentation performed by expert clinicians.

region dominating most slices. Accordingly, background suppression and weighted loss functions were explored during training to mitigate imbalance effects. Figure 5 shows the estimated frequency distribution of foreground organ classes (background excluded), highlighting the long-tail nature of several smaller organs such as pancreas and spleen.

10 Experiments Performed

A series of controlled experiments were conducted to study the behaviour of the interactive segmentation system under different design choices. These experiments focused primarily on three components: (1) the loss function, (2) the virtual user behaviour, and (3) the Sequential Interaction Memory (SIM) configuration.

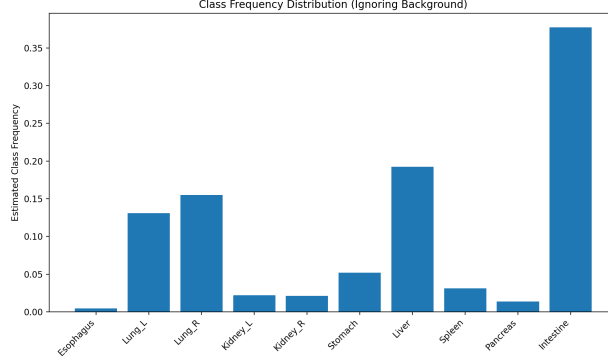


Figure 5: Estimated class frequency distribution across the dataset (background excluded). Large organs such as intestine and liver dominate the dataset, while pancreas, spleen, and esophagus appear significantly less frequently, motivating the need for weighted loss functions and background penalization.

10.1 Loss Function Experiments

We first reproduced the **Combo Loss** used in the original paper, combining Dice and Tversky losses:

$$\mathcal{L}_{\text{combo}} = \lambda_{\text{dice}} \cdot \mathcal{L}_{\text{dice}} + \lambda_{\text{tversky}} \cdot \mathcal{L}_{\text{tversky}}.$$

This served as the initial baseline.

We then evaluated additional loss formulations:

- **Weighted Cross-Entropy (CE)**: a simple re-weighted CE where the background class weight was reduced (0.5) to prevent the loss from being dominated by the large background region.
- **Dice Loss**: class-balanced overlap-based loss, independent of background frequency.
- **Dice + Weighted CE (DiceCELoss)**: a combined objective,

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{CE}}^{\text{w}} + (1 - \alpha) \cdot \mathcal{L}_{\text{dice}},$$

where CE is optionally class-weighted. This loss offered the most stable training behaviour in our experiments.

- **Weighted vs. Non-Weighted Variants**: we compared CE losses with:
 - no class weights,
 - uniform weights,
 - **custom background-only penalization** (background down-weighted, all other classes left equal).

Penalizing only the background yielded improved stability on imbalanced medical datasets.

10.2 Virtual User Behaviour

The DDG virtual user contains several configurable parameters. We experimented with:

- **Max number of clicks per round:** tested values (3, 6) to study how aggressively the virtual user corrects segmentation errors. Larger click budgets improved early refinement but showed diminishing returns.
- **Click probability decay:** observed how the decay schedule (per-round $p \leftarrow \max(0.2, p - 1/t)$) influences the density and distribution of corrective clicks.

These experiments quantified how virtual-user behaviour affects training convergence and the model’s ability to learn fine-grained refinement.

10.3 Sequential Interaction Memory (SIM) Depth

We varied the SIM depth D ($D = 3, 4$) to measure the effect of storing additional interaction rounds:

$$\text{SIM channels} = 2DN.$$

Higher SIM depth increases temporal context but also increases input dimensionality. These experiments identified an appropriate depth–complexity trade-off on our dataset.

10.4 Loss Weighting and Background Suppression

Since medical datasets often contain overwhelming background volume, we tested:

- unweighted CE,
- fully weighted CE (per-class distribution),
- **background-penalised CE only**, keeping all non-background classes unweighted.

The background-only suppression strategy produced clearer boundaries and more stable gradients without over-penalising small organs.

10.5 Normalization

Two normalization strategies were tested during training:

- **ImageNet normalization** Standard per-channel transformation used in Swin-based models:

$$x \leftarrow \frac{x - \mu_{\text{IN}}}{\sigma_{\text{IN}}}.$$

This aligns inputs with the pretrained Swin backbone distribution.

- **Per-image z-score normalization** A data-driven alternative computed for each sample:

$$x \leftarrow \frac{x/255 - \mu}{\sigma + 10^{-5}},$$

which stabilizes intensity variations across CT slices.

10.6 Summary of Findings

Overall, these experiments showed that:

- Dice + Weighted CE provided the best balance between boundary precision and class stability.
- Increasing max clicks improves refinement but saturates quickly.
- SIM depth influences stability more than absolute accuracy; moderate depth (3–4) is effective.
- Background-only penalisation gave the most reliable behaviour on our dataset.
- Z-score normalization proved out to be a better choice

11 Training Details

Both the baseline (ResNet34–UNet–ConvLSTM) and improved model (SwinUNet–ConvLSTM–SIM) were trained under a common interactive–training framework. Each epoch simulates multiple user–interaction rounds, updates the Sequential Interaction Memory (SIM), and performs a final forward pass for loss computation. Below we summarise the shared pipeline and highlight the differences between the two models.

11.1 Data Preparation and Normalization

All images were loaded from the preprocessed dataset and normalized per-sample:

$$x \leftarrow \frac{x/255 - \mu}{\sigma + 10^{-5}},$$

where μ, σ are computed across spatial dimensions. Masks were loaded as one-hot tensors and later reduced to class labels via $\arg \max$.

SIM tensors were either loaded (if available) or initialized as zero tensors of shape $(2DN, H, W)$.

11.2 Training Pipeline (Common Across Both Models)

For each batch, the following sequence was used:

1. Initialize a SIM tensor for each sample.
2. Perform an initial forward pass to obtain the first prediction.
3. For each interaction round $t = 1..D$:
 - (a) Forward pass with current SIM.
 - (b) Generate a corrective click mask using the DDG virtual user.
 - (c) Update SIM via the `update_sim` function.
4. Perform a final forward pass after all D interaction rounds.
5. Compute loss and update network weights.

This procedure exposes the model to sequential refinement behaviour, matching real interactive usage.

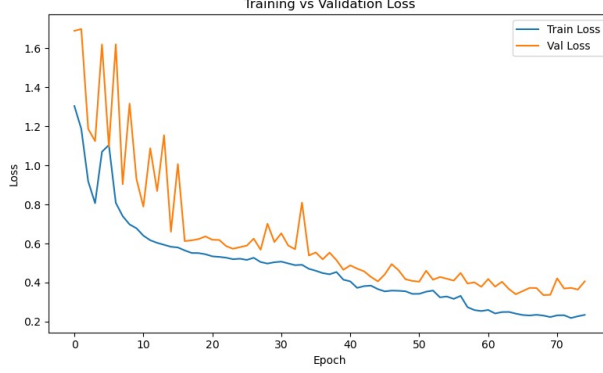


Figure 6: Training and validation loss curves for the ResNet34-UNet baseline with SIM depth $D = 3$.

11.3 Loss Functions

Unless explicitly stated in experiment sections, the primary loss used was the **Dice + Weighted Cross-Entropy loss**:

$$\mathcal{L} = \lambda_{\text{CE}} \cdot \mathcal{L}_{\text{CE}}^w + \lambda_{\text{dice}} \cdot \mathcal{L}_{\text{dice}}, \quad (\lambda_{\text{CE}} = 0.4, \lambda_{\text{dice}} = 0.6),$$

with optional class weights computed from dataset frequency.

Optimizer: AdamW with learning rate 3×10^{-4} , weight decay 10^{-5} . Scheduler: ReduceLROnPlateau (factor 0.5, patience 5). Early stopping patience: 10 epochs.

11.4 Training Differences Between the Two Models

1. Model Inputs

- **ResNet34-UNet baseline:** SIM is concatenated with the image:

$$\text{model_input} = [x \parallel \text{SIM}].$$

- **SwinUNet model:** The image and SIM are passed separately:

$$\text{model}(x, \text{SIM}),$$

because SIM is fused inside the encoder (early SIM fusion block).

2. Temporal Modeling

- **Baseline:** ConvLSTM is executed at *every decoder stage*, unrolled across D SIM states. Thus, for each round, ****multiple ConvLSTM loops**** occur at different spatial scales.
- **Improved model:** A *single* ConvLSTM operates only on the bottleneck feature e_4 . Decoder contains no recurrent computation, making training lighter.



Figure 7: Training and validation loss curves for the Swin-UNet model with SIM depth $D = 3$.

3. SIM Usage

- **Baseline:** SIM is resized and injected at all 4 decoder scales.
- **Improved:** SIM is fused *early in the encoder* (only once), influencing all Swin layers.

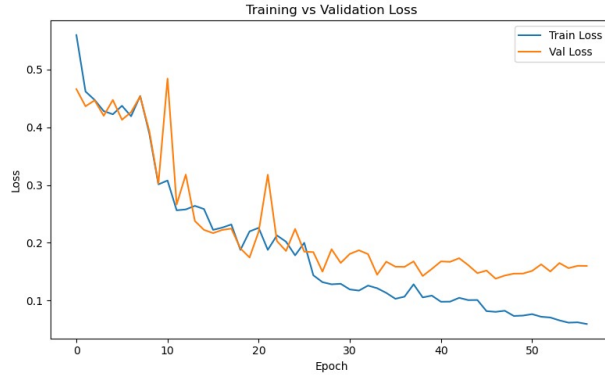


Figure 8: Training and validation loss curves for the Swin-UNet model with SIM depth $D = 4$.

4. Training Stability

- Baseline training is heavier due to multiple ConvLSTM unrollings per step.
- Swin model trains faster per iteration and is more stable to large SIM depths.

12 Results

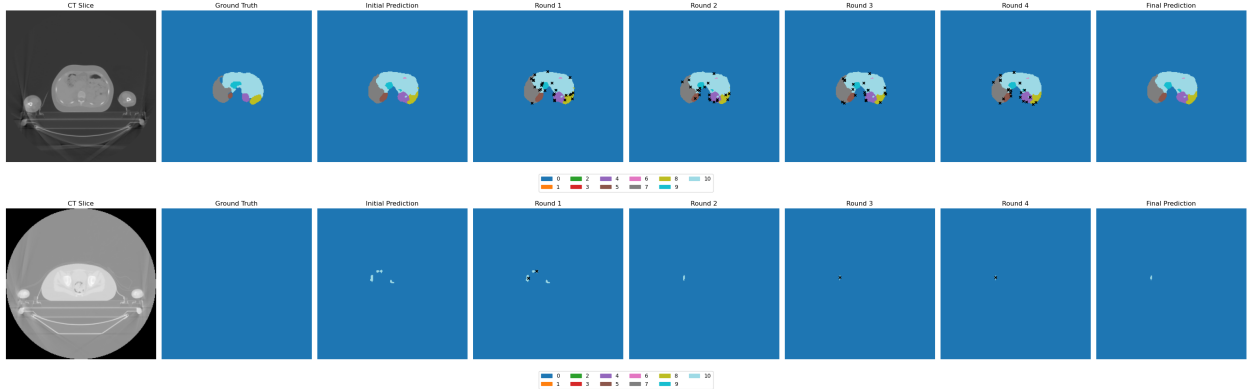


Figure 9: Interactive segmentation refinement visualized over multiple SIM rounds. Each example shows the CT slice, ground-truth mask, initial prediction, and iterative DDG virtual-user corrections. Black crosses denote simulated clicks, with the final segmentation improving progressively as SIM accumulates interaction cues.

Class	ResNet34 Baseline	Swin (3 clicks, D=3)	Swin (6 clicks, D=3)	Swin (6 clicks, D=4)
Background	0.9980	0.9976	0.9985	0.9983
Esophagus	0.4079	0.2135	0.4019	0.4009
Lung_L	0.6572	0.7571	0.8602	0.8553
Lung_R	0.7220	0.7794	0.8548	0.8535
Kidney_L	0.4505	0.4469	0.5923	0.5888
Kidney_R	0.3711	0.4834	0.5644	0.5620
Stomach	0.2890	0.4575	0.4859	0.4856
Liver	0.5526	0.7204	0.7625	0.7616
Spleen	0.2603	0.5349	0.5898	0.5886
Pancreas	0.2520	0.2819	0.3125	0.3120
Intestine	0.4579	0.5659	0.6216	0.6103
Overall Avg	0.4926	0.5671	0.6404	0.6370

Table 1: Per-class performance comparison across models and click settings.

Based on the experiments and the test-set results, we can conclude the following concise observations:

- **ResNet vs. Swin behaviour:** The baseline ResNet encoder produced stronger initial predictions, but its refinement across SIM rounds was limited. The Swin model, despite a weaker initial prediction, benefited significantly more from interactive corrections and SIM accumulation, achieving much larger round-wise improvements and ultimately outperforming ResNet by a considerable margin.
- **SIM depth:** SIM depth $D = 3$ provided the best trade-off on our dataset (overall average

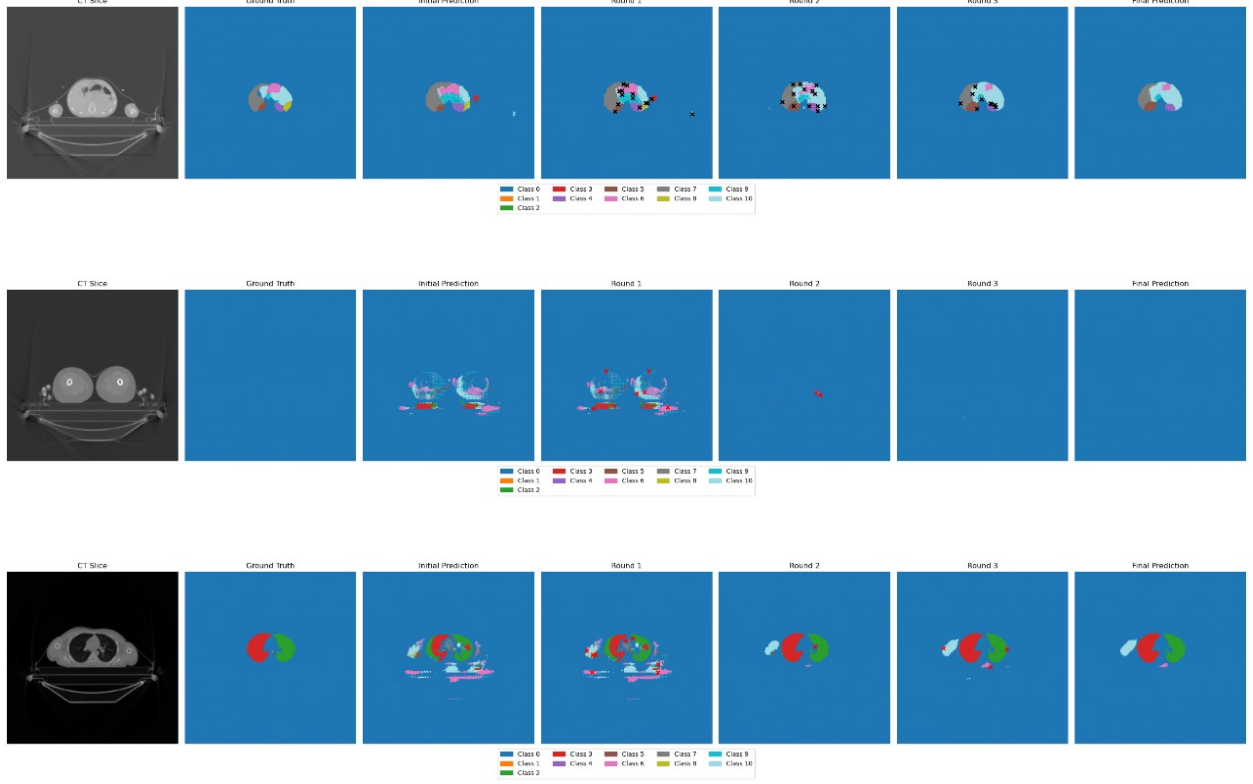


Figure 10: Model results for the baseline ResNet34-UNet with SIM depth $D = 3$. These examples illustrate segmentation quality across different slices, showing the effect of interaction accumulation through SIM.

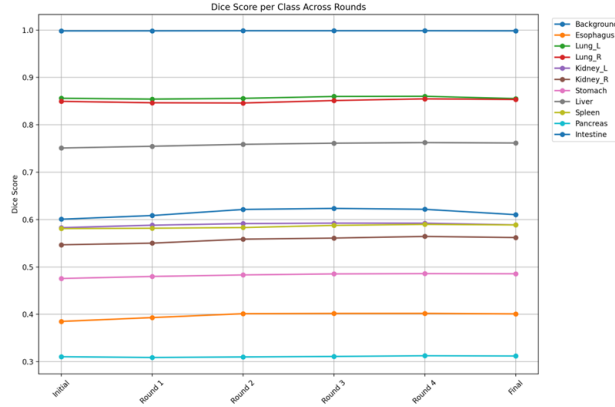


Figure 11: Per-class Dice scores for the Swin Transformer model across interactive rounds. The lungs and liver achieve the highest accuracy, while smaller structures show modest but consistent improvements with SIM-based refinement.

Dice 0.6404 for Swin, $D = 3$) compared to $D = 4$ (overall 0.6370). Hence, $D = 3$ was selected as the preferred configuration.

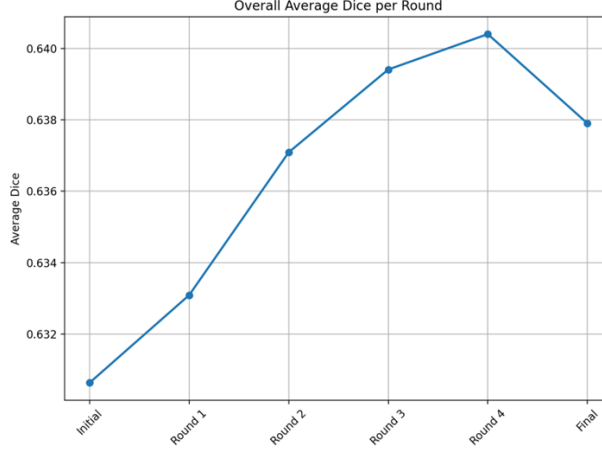


Figure 12: Overall average Dice progression for the Swin Transformer model. Interactive refinement significantly boosts the score after the first click, with diminishing but positive gains in subsequent rounds.

- **Effect of clicks:** Accuracy increases as the number of simulated clicks grows (e.g. from the non-interactive baseline to the best interactive setting), with largest gains occurring within the first 3–4 clicks and marginal or no gains thereafter (the per-round curve peaks at Round 4 and slightly decreases at final).
- **Swin transformer advantage:** The Swin-based model consistently outperformed the ResNet baseline. A concise reason: Swin’s hierarchical self-attention (shifted-window mechanism) provides stronger global context and boundary-aware features which, when combined with early SIM fusion, improves localisation of small organs and boundary consistency.
- **Top-performing classes:** Apart from the background (near-perfect), **lungs** and **liver** achieved the highest Dice scores (best per-class Dice: Lung.L ≈ 0.8602 , Lung.R ≈ 0.8548 , Liver ≈ 0.7625).
- **Best Dice and relative gains:** Comparing the Swin best configuration (max_clicks=6, $D = 3$, overall avg = 0.6404) against the ResNet baseline (overall avg = 0.4926):

$$\Delta_{\text{overall}} = 0.6404 - 0.4926 = 0.1478 \quad (\approx 30.0\% \text{ relative improvement}).$$

Additional notable per-class improvements (absolute and relative):

- Lung.L: $0.6572 \rightarrow 0.8602$ ($\Delta = 0.2030$, $\approx 30.9\%$ relative gain).
- Liver: $0.5526 \rightarrow 0.7625$ ($\Delta = 0.2099$, $\approx 38.0\%$ relative gain).
- Kidney_R: $0.3711 \rightarrow 0.5644$ ($\Delta = 0.1933$, $\approx 52.1\%$ relative gain).
- Spleen: $0.2603 \rightarrow 0.5898$ ($\Delta = 0.3295$, $\approx 126.5\%$ relative gain).
- **Challenging organs:** Small organs such as *pancreas* remain difficult (best Dice ≈ 0.3125), indicating room for further model or data improvements (e.g., stronger augmentations, focused patch training).

- The high absolute Dice for lungs and liver suggests those organs are both larger and easier to localise, and they benefit most from the combination of Swin features + interaction history.

13 Future Work

While the proposed system demonstrates strong gains with SIM-driven interaction and the Swin-based encoder–decoder design, several directions remain open for improvement:

- **Patient-consistent attention across slices:** Instead of mixing all slices from all patients during training, future systems can model attention across adjacent slices *within the same patient*. This may capture anatomical continuity and significantly improve organ boundary consistency.
- **Improved and balanced dataset:** The current dataset is highly skewed, with several organs being under-represented. A more balanced dataset, or targeted data collection for smaller structures, would reduce bias and improve per-class Dice stability.
- **SAM2-inspired prompt fusion for SIM history:** Future work could fuse SIMEncoder click maps together with probability maps into high-quality prompt embeddings using temporal multi-head attention, following ideas from SAM2. This would allow the model to leverage interaction history more effectively and produce sharper corrections.
- **Deformable attention in the decoder:** Incorporating deformable attention into the decoder could enable spatially flexible feature aggregation. This would help the model focus on fine-grained, irregular organ shapes and improve segmentation of small or complex structures.

14 References

References

- [1] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021.
- [2] Mikhailov, I., Chauveau, B., Bourdel, N., & Bartoli, A. *A deep learning-based interactive medical image segmentation framework with sequential memory*.